Datos para Machine Learning

NumPy, Pandas y exploración de datos

NumPy, Pandas y exploración de datos

Después de haber introducido en las primeras clases qué entendemos por Inteligencia Artificial y cómo se relaciona con el aprendizaje automático, ahora necesitamos detenernos en un aspecto que muchas veces queda oculto detrás de los algoritmos: **los datos**.

Antes de entrenar un modelo, necesitamos ser capaces de trabajar con la información que lo alimentará. Un algoritmo de Machine Learning puede ser sofisticado, pero si los datos están mal organizados, no sabemos interpretarlos o no somos capaces de manipularlos correctamente, difícilmente podremos construir una solución confiable.

Por eso esta clase no comienza con un algoritmo de Machine Learning. Comienza con algo más básico y, al mismo tiempo, fundamental: **aprender a representar, explorar y manipular datos utilizando Python.**

Durante la clase vamos a trabajar principalmente con dos herramientas que aparecen constantemente en proyectos de Inteligencia Artificial y análisis de datos: **NumPy** y **Pandas**. Aunque ambas trabajan con datos, no cumplen exactamente la misma función: NumPy está orientado al cálculo numérico y al trabajo eficiente con arrays, mientras que Pandas está pensado para datos estructurados — información organizada en filas y columnas. Esta diferencia parece pequeña al principio, pero se vuelve fundamental cuando pasamos de un pequeño ejemplo a un dataset real.

Los datos como materia prima de Machine Learning

Cuando hablamos de Machine Learning solemos concentrarnos en el modelo: una regresión, un árbol de decisión, KNN o una red neuronal. Sin embargo, todos esos modelos necesitan recibir información. Podemos pensar en un sistema de Machine Learning como una cadena de etapas:



En las clases posteriores vamos a estudiar con mayor profundidad las etapas que aparecen después de los datos. En esta clase nos concentraremos en las primeras dos: datos y exploración.

Supongamos que queremos desarrollar un sistema para un comercio electrónico capaz de analizar el comportamiento de sus clientes. Podríamos disponer de información como la siguiente:

cliente	edad	compras	gasto	categoría	última compra	¿compró?
C001	22	4	125.000	Ropa	10/05	Sí
C002	35	8	310.000	Electrónica	11/05	Sí
C003	29	2	78.000	Hogar	—	No

A primera vista parece una simple tabla. Pero antes de utilizarla necesitamos poder responder preguntas muy concretas: ¿cuántos clientes tenemos?, ¿cuántas variables?, ¿qué tipo de dato contiene cada columna?, ¿hay valores faltantes?, ¿existen categorías escritas de diferentes maneras?, ¿qué valores aparecen con mayor frecuencia?, ¿hay valores que parecen extraños?

Todavía no estamos entrenando ningún modelo. Estamos tratando de conocer nuestros datos — y esa etapa es fundamental.

¿Por qué necesitamos herramientas especializadas?

Python permite trabajar con números utilizando sus estructuras básicas. Podemos tener `edades = [18, 22, 35, 41, 29]` y realizar operaciones sobre ellos. Pero cuando la cantidad de datos aumenta, trabajar únicamente con listas tradicionales comienza a resultar incómodo e ineficiente.

Imaginemos ahora que tenemos 5.000 clientes y 30 variables por cliente. Necesitamos realizar operaciones como calcular promedios, filtrar clientes, seleccionar columnas, comparar valores, realizar operaciones matemáticas, buscar valores faltantes, agrupar información y calcular estadísticas. Para eso existen bibliotecas especializadas:

- **NumPy** proporciona estructuras y operaciones eficientes para cálculo numérico.
- **Pandas** agrega una estructura especialmente cómoda para representar datos tabulares y analizarlos.

Esta separación nos permite trabajar con los datos de una manera mucho más cercana a cómo se utilizan realmente en proyectos de ciencia de datos y Machine Learning.

NumPy, cuyo nombre proviene de *Numerical Python*, es una biblioteca fundamental del ecosistema científico de Python. Su estructura principal es el **array multidimensional**, representado mediante ndarray.

El ndarray

Podemos comenzar con algo muy sencillo:

```
import numpy as np


edades = np.array([18, 22, 35, 41, 29])
type(edades)
# numpy.ndarray
```

Aquí edades deja de ser una lista común de Python y pasa a ser un objeto NumPy. La importancia de esta estructura no está solamente en almacenar números: NumPy permite realizar operaciones sobre conjuntos completos de datos de manera eficiente.

```
edades + 1
# [19 23 36 42 30]
```

No necesitamos recorrer manualmente cada elemento. Esta característica es una de las razones por las que NumPy resulta tan importante para el cálculo numérico.

El concepto de dimensión

Un array puede tener una o varias dimensiones. Un array de una dimensión puede representar algo parecido a una lista:

```
x = np.array([10, 20, 30, 40])
# [10 20 30 40]
```

Pero también podemos construir una matriz de dos dimensiones:

```
X = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

Podemos consultar su forma con X.shape, que devuelve (2, 3): esto significa que tenemos **2 filas y 3 columnas**.

Shape y representación de los datos

Supongamos que tenemos información de 1.000 clientes y disponemos de 5 características para cada uno. En Python, X.shape podría devolver (1000, 5), lo que nos dice inmediatamente algo importante: tenemos 1.000 observaciones y 5 variables. Esta relación entre la estructura de los datos y la forma

matemática en que representamos un problema de Machine Learning aparecerá continuamente durante las próximas clases.

Creación de arrays

NumPy proporciona diferentes formas de crear arrays: directamente, generando secuencias, generando valores equiespaciados, o inicializados con ceros o unos.

```
np.array([1, 2, 3, 4])

np.arange(0, 10)          # [0 1 2 3 4 5 6 7 8 9]
np.linspace(0, 1, 5)      # 5 valores entre 0 y 1

np.zeros((3, 4))          # matriz de ceros
np.ones((2, 3))           # matriz de unos
```

Estas operaciones son frecuentes cuando necesitamos preparar estructuras numéricas para cálculos posteriores.

Tipos de datos

Un array NumPy posee un tipo de dato, que podemos consultar con x.dtype. Por ejemplo, podríamos obtener int64 si los valores son enteros, o un tipo de punto flotante si trabajamos con números reales como np.array([1.5, 2.3, 4.8]).

La cuestión puede parecer técnica, pero tiene consecuencias prácticas: en Inteligencia Artificial trabajamos constantemente con números, los modelos realizan operaciones matemáticas sobre ellos y, por lo tanto, conocer cómo están representados nuestros datos es importante.

Indexing: acceder a elementos

Podemos acceder a un elemento particular utilizando su índice. Recordemos que Python comienza a contar desde cero:

```
edades = np.array([15, 22, 18, 30, 12, 45])
edades[0]    # 15  (índice 0)
edades[-1]   # 45  (índice negativo: último elemento)
```

También podemos indicar fila y columna cuando tenemos dos dimensiones, con la forma X[fila, columna]:

```
X = np.array([[10, 20, 30], [40, 50, 60]])
X[0, 1]   # 20
X[1, 2]   # 60
```

Slicing

El **slicing** permite seleccionar una parte de un array con la estructura array[inicio:fin], donde el índice inicial se incluye y el final no.

```
edades = np.array([15, 22, 18, 30, 12, 45])
edades[1:4]    # [22 18 30]
edades[::2]    # [15 18 12]   (un elemento cada dos posiciones)
```

El slicing es especialmente útil porque nos permite seleccionar partes de nuestros datos sin necesidad de crear manualmente nuevas estructuras.

Boolean indexing y máscaras

Una de las características más importantes de NumPy es la posibilidad de utilizar condiciones para seleccionar datos. Si escribimos edades >= 18 obtenemos una **máscara booleana**: un array que indica, para cada elemento, si cumple o no la condición.

```
edades = np.array([15, 22, 18, 30, 12, 45])
edades >= 18
# [False True True True False True]

edades[edades >= 18]
# [22 18 30 45]
```

No estamos buscando manualmente qué valores cumplen una condición: estamos expresando una condición y dejando que NumPy seleccione los elementos correspondientes. Esto es extremadamente importante para el análisis de datos, y también podemos combinar condiciones con & (AND) y | (OR):

```
edades[(edades >= 18) & (edades < 30)]   # entre 18 y 30
edades[(edades < 18) | (edades > 40)]    # menores de edad o mayores de 40
```

Modificación de elementos y vectorización

Los arrays también pueden modificarse, incluso a partir de una máscara:

```
edades = np.array([15, 22, 18, 30])
edades[0] = 16
edades[edades < 18] += 1
```

Una característica fundamental de NumPy es la **vectorización**: aplicar una operación sobre todo el array de una manera eficiente, sin escribir un for.

```
precios = np.array([100, 200, 300, 400])
precios * 1.10
# [110. 220. 330. 440.]
```

Esta forma de trabajar resulta particularmente importante en IA porque gran parte de los algoritmos implica realizar operaciones matemáticas sobre grandes cantidades de valores.

Broadcasting

El **broadcasting** amplía esta idea. Si tenemos una matriz X y hacemos X + 100, NumPy interpreta que queremos sumar 100 a cada elemento, sin necesidad de construir otra matriz llena de 100.

```
X = np.array([[10, 20, 30], [40, 50, 60]])
X + 100
# [[110 120 130]
#  [140 150 160]]
```

NumPy realiza automáticamente la operación compatible entre las dimensiones. Este mecanismo permite escribir operaciones matemáticas compactas y eficientes.

Agregaciones

Muchas veces no queremos observar todos los valores individualmente: queremos obtener un resumen.

```
np.sum(edades)     # suma total
np.mean(edades)    # promedio
np.min(edades)     # mínimo
np.max(edades)     # máximo
np.argmin(edades)  # posición del mínimo
np.argmax(edades)  # posición del máximo
```

min responde: ¿cuál es el valor mínimo? **argmin** responde: ¿en qué posición está el mínimo? La diferencia es importante.

Operaciones por ejes

Cuando trabajamos con matrices aparece el concepto de **eje** (axis). Sobre la matriz X del ejemplo anterior, podemos pedir una suma por columnas o por filas:

```
np.sum(X)          # 210  (total)
np.sum(X, axis=0)    # [50 70 90]   (suma por columnas)
np.sum(X, axis=1)    # [60 150]     (suma por filas)
```

Comprender los ejes será importante cuando trabajemos con matrices de características y cálculos numéricos más complejos.

Matrices, transpuesta y producto matricial

Una matriz puede transponerse — intercambiar filas por columnas — mediante X.T. También podemos realizar operaciones fundamentales de álgebra lineal: el producto escalar entre dos vectores, y el producto matricial.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.dot(a, b)     # 1*4 + 2*5 + 3*6 = 32
```

```
A @ B           # producto matricial
np.matmul(A, B)   # equivalente
```

Estas operaciones no son solamente matemática abstracta: las redes neuronales, por ejemplo, realizan continuamente productos entre matrices y vectores. Por eso aprender NumPy no significa simplemente aprender una biblioteca: significa comenzar a familiarizarnos con las operaciones numéricas que aparecen debajo de muchos algoritmos de Inteligencia Artificial.

De NumPy a Pandas

Hasta ahora estuvimos trabajando principalmente con números organizados en arrays. Pero nuestros datasets reales suelen tener una estructura diferente: identificadores, números y categorías combinados, como en esta tabla de e-commerce:

Cliente	Edad	Compras	Gasto	Categoría
C001	22	4	125.000	Ropa
C002	35	8	310.000	Electrónica
C003	29	2	78.000	Hogar

Necesitamos una herramienta que permita trabajar cómodamente con esta estructura. Ahí aparece Pandas.

Series y DataFrame

Una **Series** puede pensarse como una columna de datos, donde cada valor posee un índice:

```
import pandas as pd
edades = pd.Series([22, 35, 29, 41])
```

El **DataFrame** es probablemente la estructura de Pandas que más vamos a utilizar: podemos imaginarlo como una tabla completa, con filas, columnas e índices.

```
datos = {
    'edad': [22, 35, 29],
    'compras': [4, 8, 2],
    'gasto': [125000, 310000, 78000]
}
df = pd.DataFrame(datos)
```

Esta estructura se adapta perfectamente a la representación habitual de un dataset de Machine Learning.

Leer un archivo CSV

En un proyecto real normalmente no vamos a escribir los 40.000 registros manualmente dentro del código: los datos estarán almacenados en un archivo. Uno de los formatos más habituales es CSV, y Pandas permite leerlo directamente con `pd.read_csv("dataset.csv")`.

Existe un detalle importante: **el delimitador**. Un CSV puede usar coma o punto y coma; si nuestro archivo utiliza punto y coma, podemos indicarlo:

```
df = pd.read_csv("dataset.csv", sep=";")
```

Primera inspección

Una vez cargado un dataset, no deberíamos comenzar inmediatamente a modificarlo. Primero debemos observarlo:

```
df.head()    # primeras filas
df.tail()    # últimas filas
df.shape     # (40000, 12) →40000 filas, 12 columnas
df.info()    # tipos de dato, nulos y estructura general
```

df.info() nos permite empezar a responder preguntas importantes: ¿cuántas variables tenemos?, ¿qué tipo de dato contiene cada una?, ¿hay valores faltantes?, ¿qué columnas parecen numéricas y cuáles categóricas? La exploración comienza a convertirse así en un proceso sistemático.

Seleccionar columnas y filtrar filas

Podemos seleccionar una o varias columnas, y filtrar registros mediante condiciones — de manera conceptualmente similar al boolean indexing de NumPy, pero directamente sobre estructuras tabulares con nombres de columna:

```
df["gasto"]              # una columna
df[["edad", "gasto"]]      # varias columnas

df[df["edad"] >= 18]
df[(df["edad"] >= 18) & (df["gasto"] > 100000)]
```

Estadísticas descriptivas y percentiles

Una vez que conocemos la estructura, podemos obtener estadísticas con df.describe(), que genera un resumen de las variables numéricas: count, mean, std, min, 25%, 50%, 75% y max.

mean	min	25%	50%	75%	max
152.000	500	50.000	110.000	180.000	2.500.000

Frente a estos números aparece una pregunta interesante: ¿por qué existe tanta diferencia entre el valor máximo y la mediana? ¿Tenemos clientes que realmente gastaron 2.500.000, o hay un problema en los datos? Esta pregunta nos llevará directamente a la próxima clase, sobre calidad de datos y valores extremos.

Los **percentiles** permiten comprender mejor cómo se distribuyen los datos: el percentil 50 corresponde a la mediana, el 25% de los datos se encuentra por debajo del primer cuartil y el 75% por debajo del tercer cuartil.

value_counts()

Cuando tenemos variables categóricas, una herramienta especialmente útil es df["categoria"].value_counts(), que nos permite conocer la frecuencia de cada categoría:

Categoría	Frecuencia
Ropa	1800
Electrónica	1400
Hogar	950
Deportes	650

Esto permite detectar situaciones interesantes: si esperábamos una distribución equilibrada y una categoría concentra casi todos los registros, debemos investigarlo. También podemos descubrir categorías inesperadas, como "Ropa", "ropa" y "ROPA": desde el punto de vista de Python son tres valores diferentes; desde el punto de vista del negocio, quizás representen la misma categoría. Este tipo de problema será central cuando estudiemos calidad de datos.

groupby()

Otra operación fundamental es agrupar información. Si queremos conocer cuánto gastan en promedio los clientes de cada categoría, podemos escribir df.groupby("categoria")["gasto"].mean():

Categoría	Gasto promedio
Electrónica	210.000
Hogar	125.000
Ropa	110.000
Deportes	95.000

Ya no estamos observando clientes individualmente: estamos analizando **grupos**. Podemos utilizar otras agregaciones como .sum() o .max(). Esto será muy útil para explorar el comportamiento del e-commerce antes de comenzar a construir modelos.

Valores faltantes

Uno de los primeros problemas que debemos buscar en un dataset real son los valores faltantes. NaN representa que no tenemos un valor disponible. Podemos detectarlos con df.isna(), y normalmente queremos conocer cuántos existen por columna:

```
df.isna().sum()
# edad       1
# gasto      0
# categoria  3
```

¿Qué hacemos con un valor faltante? Esta pregunta no tiene una única respuesta: podemos eliminar registros con df.dropna() (lo que implica perder información), o reemplazar valores con df.fillna(...) utilizando algún criterio, como la media o la mediana de una variable numérica.

En determinados tipos de datos, especialmente cuando existe un orden temporal, podemos utilizar **interpolación** para estimar valores intermedios, mediante df["ventas"].interpolate(). Pero la herramienta no decide por nosotros si la interpolación tiene sentido: la técnica debe ser compatible con la naturaleza de los datos.

Fechas y texto

Los datos de e-commerce suelen contener fechas. Pandas permite convertirlas a un formato temporal apropiado y extraer componentes:

```
df["fecha"] = pd.to_datetime(df["fecha"])
df["fecha"].dt.year
df["fecha"].dt.month
df["fecha"].dt.day
```

Esto permite transformar una fecha en información que posteriormente puede resultar útil para el análisis — por ejemplo, para saber si las ventas aumentan durante determinados meses.

Los datasets también pueden contener información textual. Pandas permite operar sobre strings, por ejemplo convirtiendo a minúsculas o eliminando espacios:

```
df["categoria"].str.lower()
df["categoria"].str.strip()
```

Esto resulta especialmente útil cuando encontramos categorías inconsistentes como "Ropa", "ropa", " Ropa " o "ROPA". Antes de utilizar estos datos podemos necesitar normalizarlos.

NumPy y Pandas: dos herramientas diferentes

NumPy está orientado al cálculo numérico y al trabajo con arrays. **Pandas** está orientado al trabajo con datos estructurados y tabulares.

	NumPy	Pandas
Estructura	arrays (ndarray)	DataFrames / Series
Enfoque	cálculo numérico	análisis tabular

No son herramientas independientes: Pandas utiliza NumPy internamente en muchas de sus operaciones, y ambas forman parte del mismo ecosistema científico de Python. En un proyecto de Machine Learning es habitual utilizar ambas — Pandas para cargar y explorar un dataset, y NumPy para realizar determinadas operaciones numéricas. Posteriormente, otras bibliotecas como Scikit-Learn utilizarán estructuras compatibles con este ecosistema para entrenar los modelos.

Primer contacto con el dataset del proyecto

Ahora podemos llevar todo esto al proyecto transversal. Supongamos que recibimos nuestro archivo ecommerce.csv. Nuestra primera tarea no debería ser entrenar un modelo, sino hacer una primera exploración:

```
import pandas as pd
import numpy as np

df = pd.read_csv("ecommerce.csv")
df.head()
df.shape
df.info()
df.describe()
df["categoria"].value_counts()
df.isna().sum()
```

La importancia de no tocar los datos demasiado rápido

Existe una tentación muy común cuando se comienza a trabajar con datasets: encontrar algo extraño y modificarlo inmediatamente. Supongamos que encontramos un registro con edad = 250. Podríamos pensar de inmediato: "esto está mal, lo elimino". Pero todavía no sabemos qué ocurrió: puede ser un error de carga, una variable que no representa edad, un problema de desplazamiento de columnas, o un registro realmente corrupto.

Del dataset RAW al dataset CLEAN

En nuestro proyecto vamos a trabajar con una idea particularmente importante. Partiremos de un dataset RAW — una versión que representa datos tal como podrían encontrarse en un escenario real, incluyendo problemas que tendremos que detectar y analizar. Nuestro objetivo posterior será construir un dataset CLEAN, preparado para ser utilizado en Machine Learning:

RAW	Exploración	Detección de problemas	Decisiones	Limpieza	CLEAN

Introdujimos NumPy como herramienta para el cálculo numérico y trabajamos con su estructura fundamental, el ndarray: interpretamos dimensiones y shape, creamos arrays, consultamos tipos de datos y accedimos a elementos mediante indexing y slicing. También utilizamos máscaras booleanas para filtrar información y vimos cómo las operaciones vectorizadas y el broadcasting permiten realizar cálculos sobre grandes cantidades de datos sin necesidad de recorrerlos manualmente.

Después pasamos a Pandas, donde encontramos estructuras más adecuadas para representar datasets tabulares: trabajamos con Series y DataFrame, aprendimos a cargar archivos CSV y a realizar una primera inspección mediante head, tail, shape e info. A partir de allí comenzamos a explorar los datos mediante selección de columnas, filtros, estadísticas descriptivas, percentiles, frecuencias y agrupaciones con value_counts y groupby. También introdujimos la detección de valores faltantes, el manejo básico de fechas y algunas operaciones sobre texto.

Pero quizás la idea más importante de la clase no sea una función concreta de NumPy o Pandas. La idea central es que **los datos no deben utilizarse a ciegas**. Antes de entrenar un modelo debemos conocer qué representan, cómo están organizados y qué problemas podrían contener. Por eso nuestro recorrido comienza con:

Datos	Exploración	Comprensión	Preparación	Modelo